

The Untraceable Model

A Cordwell Home and Hardware post-incident review

Teams of 3 · 30 to 45 minutes · Covers slides 1 to 21: MLflow architecture, local setup, parameters, metrics, tags, autolog, artifacts, and model logging

The Setup

It is Monday morning at Cordwell Home and Hardware. Over the weekend, the model that flags customer reviews for safety-team escalation quietly missed a review describing a table saw guard that detaches during use. The safety team caught it two days late through a manual audit, and now they are in your team's channel asking one question:

“Which training run produced the model that is escalating reviews right now, and can you prove it?”

Your team pulls up everything that exists. It is not much:

- `final_v3_REAL.ipynb` on Priya's laptop, last edited three weeks ago, which she is “pretty sure” is the one
- `metrics_new2.csv` in a shared drive, with columns `acc`, `rec`, and `f1_maybe`, no dates, no run identifiers
- A Teams message from Marcus, six weeks old: “deployed the good one, recall was way better”
- A model file named `model_FINAL_fixed.pkl` whose training code nobody can locate

Nobody did anything careless. Everyone did what most teams do by default. Today your team runs the post-incident review, and your job is not to assign blame to a person. Your job is to assign every failure to the missing piece of infrastructure that would have prevented it, then rebuild the record properly.

Ground rule for the whole exercise: blame components, not people. Every answer of the form “Priya should have been more careful” gets rewritten as “the tracking server would have captured that automatically.”

Your Team

Assign one role per person. Rotate roles at the start of each act so everyone holds each role once.

- **Incident Lead.** Keeps the team on the clock, reads each prompt aloud, and calls the final answer when time is short.
- **Scribe.** Writes the team's answers on this handout. If it is not written down, it did not happen, which is fitting, because that is the entire lesson of this module.
- **Challenger.** Must push back on at least one team decision per act with a “what breaks if we are wrong” argument. A team that agrees instantly on everything is a team that has not found the interesting part yet.

Act 1 · Anatomy of the Mess

8 minutes · Slides 4 to 7

Slide 5 gives you four MLflow components: the **Tracking Server**, the **Client APIs**, the **Artifact Store**, and the **Model Registry**. As a team, list at least four specific questions the safety team could reasonably ask that your current evidence cannot answer. For each one, name the component that would have answered it and say in one sentence how.

Question the safety team asked that nobody could answer	MLflow component that would have answered it, and how

Quick-fire finisher, discuss for one minute: the whole fix for Cordwell today is a single line, `mlflow.set_tracking_uri("sqlite:///mlflow.db")`, plus the discipline to log. What is the strongest argument someone on your team could make for NOT adopting it, and does that argument survive contact with this incident?

Act 2 • The Great Sort

12 minutes • Slides 8 to 12 and 17 to 19

Digging through the wreckage of `final_v3_REAL.ipynb`, your team recovers the sixteen items below. If Cordwell had been logging properly, every one of them would have a correct home in MLflow. Classify each item as exactly one of: **Parameter**, **Metric**, **stepped**, **Metric**, **final**, **Tag**, or **Artifact**. Write a one-line justification for any item your team debated.

Scoring, if your facilitator is running it competitively: one point per correct call. Items 5, 7, 10, and 11 are worth two points, and only with a written justification. There is at least one item on which reasonable engineers disagree, and your justification matters more than your label.

#	Item found in the wreckage	Your call	Why (one line)
1	<code>C = 0.1</code>		
2	<code>recall = 0.8308</code> on the holdout set		
3	<code>train_f1</code> recorded at the end of every epoch		
4	<code>class_weight = "balanced"</code>		
5	<code>seed = 42</code>		
6	<code>owner = ml-team</code>		
7	<code>dataset_version = cordwell_v2</code>		
8	<code>confusion_matrix.png</code>		
9	<code>latency_p95</code> measured on the final model		
10	the prompt template handed to an LLM judge		

#	Item found in the wreckage	Your call	Why (one line)
11	MODEL = "logreg" held in a config variable		
12	per_review_predictions.csv with ground truth		
13	training_config.json		
14	experiment_type = regularization_sweep		
15	validation accuracy recorded at every epoch		
16	the fitted classifier itself		

Tie-breaker, answer as a team: mid-run, someone calls `mlflow.log_param("C", 0.5)` for a run where C was already logged as 0.1. What exactly happens, and why is that behavior a feature rather than a bug? Then answer the same question for logging C a second time with the *same* value.

Act 3 • Fix the Record

10 minutes • Slides 13 and 20 to 21

A well-meaning teammate has already drafted the “never again” logging code for the retrained Cordwell review classifier, a fine-tuned transformer. It contains three bugs of increasing subtlety. Two will bite this week on the cohort's own hardware and library versions. One will bite in a few months. Find all three, and write the corrected call below the snippet.

```
import mlflow

mlflow.set_tracking_uri("sqlite:///mlflow.db")
mlflow.set_experiment("cordwell_escalation_triage")

with mlflow.start_run(run_name="transformer_v1"):
    mlflow.transformers.log_model(
        transformers_model={
            "model": model.to("cuda"),
            "tokenizer": tokenizer,
        },
        artifact_path="model",
        registered_model_name="cordwell-review-classifier",
    )
```

Bug 1, breaks today on your Mac:

Bug 2, warns today, breaks on a future MLflow release:

Bug 3, the subtle one, costs you six months from now:

Your corrected call, written out in full:

Then settle one last argument. The same teammate says: “Why are we hand-writing any of this? Slide 13 showed `mlflow.sklearn.autolog()` capturing 14 parameters, 7 metrics, and the model with zero manual calls. Autolog everything, always.” As a team, decide where autolog genuinely ends for the Cordwell escalation workflow, and name at least two things from this module that it will never capture for you.

Debrief

6 minutes, whole room

Your Incident Lead reports out, in under 90 seconds:

- The single item from the Great Sort your team argued about longest, and where you landed.
- One question from Act 1 that you found genuinely uncomfortable, because your own real projects could not answer it either.
- Your corrected Act 3 call, read aloud.

Done When

- Act 1 table has at least four rows, each naming a component and a mechanism.
- All sixteen items in the Great Sort carry a call, and items 5, 7, 10, and 11 carry a written justification.
- All three Act 3 bugs are identified and the corrected call is written out in full.
- The Challenger has pushed back at least once per act, and the Scribe wrote down what happened to that pushback.